目录

第一章 引言 ..... 第 1 页
第三章 架构设计与选型 ..... 第 2 页
第四章 详细功能阐述 ..... 第 5 页
第五章 安装与配置 ..... 第 8 页
第六章 操作使用 ..... 第 9 页
第七章 数据库、字典与术语 ..... 第 12 页
第八章 附录 ..... 第 14 页

2.2 主要功能

（一）仿真环境构建：基于 Isaac Gym 搭建 Go2 仿真场景，支持机器人出生点与朝向的随机化，

以及障碍物与目标点的随机布置，使训练数据覆盖多样的初始条件。

（二）安全指标计算：在底层仿真环境中实时计算 avoid_metric（避障度）与

reach_metric（到达度），作为后续约束信号的基础。当距离障碍物或边界过近时，

avoid_metric 显著下降；当接近目标时，reach_metric 上升。

（三）高层观测构造：将安全指标与目标、障碍、边界的激光雷达分箱特征组合为 40 维观测向量，

为高层策略提供充分且低维的决策信息。

（四）约束信号转换：将底层指标转换为到达代价 g(s) 与安全约束 h(s)，使训练目标可由约束优化统一表达，

而非简单的标量奖励。

（五）策略训练：实现 Reach-Avoid PPO，以 g(s)/h(s) 经广义优势估计（

GAE）合成价值目标，更新高层策略参数。训练过程统计每轮成功率与执行成本，并支持断点续训。

（六）回放与评测：提供可视化回放脚本观察策略行为，提供轨迹导出脚本将评测结果保存为

JSON，便于离线统计成功率、碰撞率与执行成本。

（七）成果展示：提供基于 GitHub Pages 的静态网页，汇总算法原理、训练曲线与演示内容，

便于对外交流。

2.3 运行流程概览

典型工作流为：配置环境参数 → 启动训练（train_reach_avoid.py）

得到高层策略检查点 → 使用回放脚本（play_reach_avoid.py）观察行为

→ 使用评测脚本（test_reach_avoid.py）导出轨迹并统计指标 → 必要时绘制训练曲线（

plot_training_results.py）与场景布局（plot_env_layout.

py）。各步骤的输入输出均以文件传递。

第三章 架构设计与选型

3.1 总体架构

系统采用双层控制结构。高层策略输出速度命令，冻结的低层运动策略将速度命令转化为 12

个关节的目标动作，最终驱动 Go2 本体在仿真中运动。训练阶段，环境不返回标量奖励，

而是返回安全指标；高层环境将其转换成 g(s) 与 h(s)，经 Reach-Avoid

GAE 合成价值目标，用于更新高层策略。高层每决策一步，低层连续执行 5 步，形成分层时间抽象。

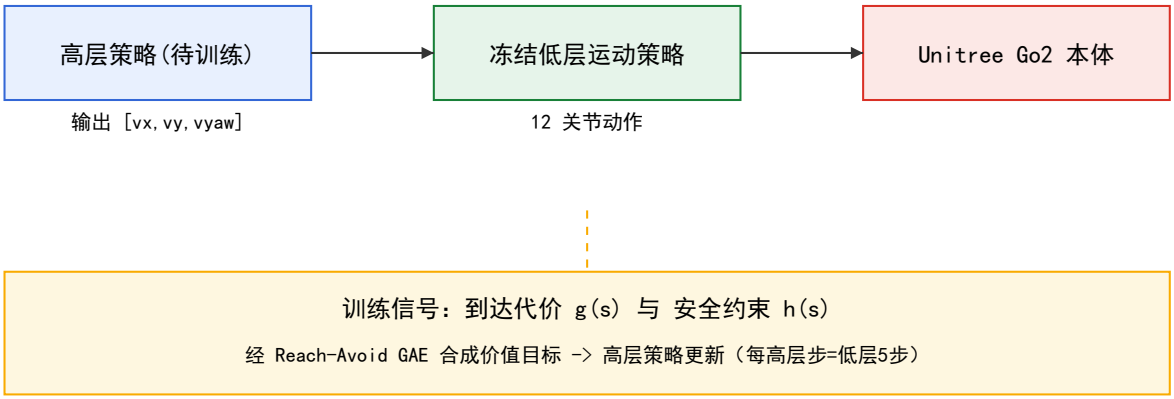


图3-1 分层控制闭环架构

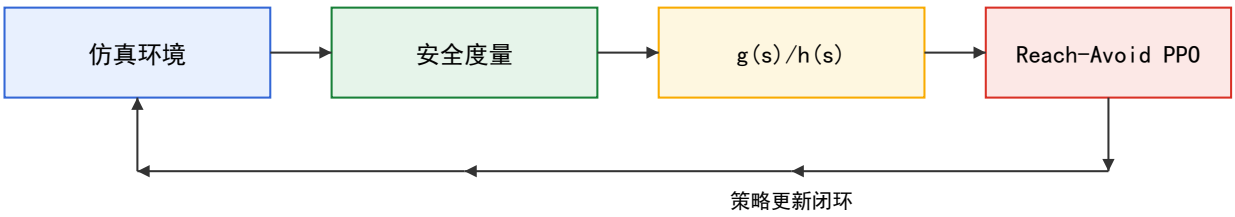


图3-2 训练信号数据流

3.2 分层时间抽象

分层抽象是本软件的关键设计。低层策略以较高频率（每仿真步）运行，保证机器人稳定行走与抗扰；

高层策略以较低频率（每 5 个低层步）运行，只在“速度命令”这一语义清晰的接口上决策。

这种时间抽象降低了决策频次，使高层策略能在更长的视野上规划到达路径，同时低层始终维持安全的基本运动。

3.3 技术选型

仿真后端选用 Isaac Gym Preview 4，因其提供大规模并行物理仿真，可在单卡上同时采样大量环境，

显著提升强化学习的采样效率；底层封装沿用 legged_gym 框架；训练框架基于

rsl_rl 的 PPO 实现，并扩展出 Reach-Avoid 专用分支；深度学习框架为

PyTorch；全部功能以 Python 实现，便于科研人员阅读与复现。

采用分层而非端到端策略的原因：低层运动控制已由冻结策略保证稳定行走，高层只需在“速度命令”这一低维、

语义清晰的接口上决策，能够显著提升样本效率，并将底层不安全动作隔离在冻结策略之内，从结构上增强安全性。

3.4 目录结构与职责

目录/文件	职责
legged_gym_go2/legged_gym/envs/go2/	Go2 仿真环境、配置与安全指标计算
.../go2/go2_env.py	底层环境，扩展 _compute_safety_metrics()
.../go2/high_level_navigation_env.py	高层观测构造与 g(s)/h(s) 转换
.../go2/hierarchical_go2_env.py	分层封装（高层1步=低层5步）
.../go2/go2_config.py	场地、障碍、目标等关键参数集中定义
legged_gym_go2/legged_gym/scripts/	训练、回放、评测与绘图脚本
rsl_rl/rsl_rl/algorithms/reach_avoid_ppo.py	Reach-Avoid PPO 核心算法
rsl_rl/rsl_rl/algorithms/ppo.py	基础 PPO 实现（被复用）
rsl_rl/rsl_rl/modules/actor_critic.py	策略网络与价值网络
rsl_rl/rsl_rl/storage/rollout_storage.py	轨迹回放缓冲

rsl_rl/rsl_rl/runners/on_policy_runner.py	训练主循环与日志
webui/	项目展示静态网页

high_level_navigation_env.py 将底层指标转换为到达代价

g(s) 与安全约束 h(s)：当机器人抵达目标附近时 g(s) 取较小的 g_target（如 -300），距离越远代价越高（由 g_distance_scale 控制）；当处于安全区域时 h(s) 取 h_safe-（如 -300），发生碰撞或越界时取 h_unsafe-（如 300）。同时构造 40 维高层观测：8 维基础特征、16 维目标激光雷达分箱特征、16 维障碍/边界激光雷达分箱特征。

3.5 模块依赖关系

高层环境（high_level_navigation_env）依赖底层环境（go2_env）提供的安全指标；分层环境（hierarchical_go2_env）装载冻结的低层策略，向上对高层策略暴露速度命令接口；训练脚本依赖reach_avoid_ppo 与 actor_critic 完成策略优化；评测与绘图脚本依赖训练产出的检查点与日志。各模块通过Python 导入与配置文件耦合，

无外部服务依赖。

第四章 详细功能阐述

4.1 环境模块

go2_env.py 在底层 Go2 仿真环境中扩展 _compute_safety_metrics()，

产出 avoid_metric 与 reach_metric，并随机化机器人出生点与朝向，

使策略对初始条件具有鲁棒性。该模块还负责物理步进、碰撞检测与回合终止判断。



高层观测 o_t ∈ R^40 = [基础 | 目标lidar | 障碍/边界lidar]

动作 a_t = [vx, vy, vyaw] ∈ R^3，由高层策略输出，经冻结低层策略映射到 12 关节

图4-1 高层观测空间构成（40维）

hierarchical_go2_env.py 装载冻结的低层策略，实现分层时间抽象：

高层输出一个速度命令后，低层连续执行 5 步，再回到高层决策。该模块对上层屏蔽了关节级控制细节，

仅暴露速度命令接口。

go2_config.py 以配置类集中定义场地尺寸（12×12 米）、障碍球数量（

14）、目标半径（0.5 米）、训练轮数（1500）等关键参数，便于实验复现与消融。

4.2 算法模块

reach_avoid_ppo.py 是本软件核心。其中 _calculate_reach_gae()

以 g(s)/h(s) 计算广义优势估计，ReachAvoidBuffer 存储含约束信号的轨迹，

ReachAvoidPPO 类据此更新高层策略。该方法把“到达”与“避障”统一为带约束的策略优化问题，

区别于普通以奖励最大化为目标的 PPO。

actor_critic.py 定义多层感知机（MLP）策略网络与价值网络，隐含层均为

512×4，动作分布采用对角高斯，适用于连续动作空间。actor_critic_recurrent.

py 提供带循环单元（如 GRU/LSTM）的可选版本，用于需要处理部分可观信息的场景。

rollout_storage.py 管理训练轨迹的收集与批量采样；on_policy_runner.

py 组织训练循环，负责任务创建、日志写入、模型保存与评估调度，并支持从检查点恢复训练。

4.3 训练与评测脚本

train_reach_avoid.py 为训练主循环，按配置创建环境、初始化算法、

迭代更新策略，并在每轮统计成功率与执行成本，将检查点保存至 logs/ 目录，支持断点续训。

play_reach_avoid.py 用于可视化回放：加载训练好的高层策略，在仿真中运行并记录画面，

便于直观检查导航行为是否安全、是否绕障、是否抵达目标。

test_reach_avoid.py 用于批量评测：在多种随机初始条件下运行策略，

将轨迹与指标导出为 JSON 文件，便于离线统计成功率、碰撞率与平均执行成本。

plot_training_results.py 与 plot_env_layout.

py 分别读取日志与配置，绘制训练曲线（成功率、成本随轮数变化）与场景布局（场地、障碍、

目标分布），辅助结果分析。

4.4 工具与展示模块

utils 下提供 helpers（通用辅助）、terrain（地形相关）、task_registry（任务注册）等模块；webui 为 GitHub Pages 静态站点，展示算法原理、训练曲线与演示视频，便于成果对外交流与评审材料补充。

Isaac Gym 需从 NVIDIA 官方渠道获取 Preview 4 版本并安装其 Python 包；PyTorch 与 CUDA 按官方指引安装；本仓库的 rsl_rl 与 legged_gym 以可编辑方式安装（pip install -e）。安装完成后，可在终端中导入相关模块验证环境。

第五章 安装与配置

5.1 硬件与软件环境

开发硬件建议为 x86_64 兼容计算机，配备 NVIDIA GPU（训练用，显存不小于 8GB、内存不小于 32GB、磁盘不小于 100GB。运行硬件为 Unitree Go2 四足机器人本体及机载计算单元。

软件环境：操作系统为 Ubuntu 20.04 或 22.04；Python 3.8 及以上；PyTorch 1.13 及以上；CUDA 11.7；Isaac Gym Preview 4；依赖库 rsl_rl 与 legged_gym。实机部署另需 Unitree 官方 SDK 与机载 Linux。

5.2 依赖获取与安装

5.3 关键配置参数

go2_config.py 集中定义导航任务的关键参数。下表列出主要参数及其在本项目中的取值，用户可依实验阶段调整。

参数	含义	本项目取值	说明
field_size	场地边长	12 m	12×12 米方形场地
n_obstacles	障碍球数量	14	随机布置于场地内
target_radius	目标半径	0.5 m	进入即判定到达
g_target	到达目标代价	-300	抵达目标时取该值
g_distance_scale	距离缩放系数	100	控制距离对代价的权重
h_safe	安全时代价	-300	未碰撞/未越界
h_unsafe	不安全时代价	300	碰撞或越界
obs_dim	高层观测维度	40	基础8+目标16+障碍/边界16
act_dim	动作维度	3	vx, vy, vyaw
train_iters	训练轮数	1500	本项目实际训练量

5.4 观测与动作空间

高层观测为 40 维向量，组成见图4-1：前 8 维为基础特征（含机器人位姿、速度、与目标相对位置等）；中间 16 维为目标方向的激光雷达分箱特征；后 16 维为障碍/边界的激光雷达分箱特征。

动作空间为 3 维连续向量 [vx, vy, vyaw]，分别表示前进/后退线速度、左/右平移速度、偏航角速度，范围由配置约束。

激光雷达分箱（lidar bin）指将机器人周围空间按角度划分为若干扇区，取各扇区内最近障碍（

或边界”的距离作为该扇区的特征值。这种表示既压缩了原始点云维度，又保留了绕障所需的方位信息，

适合作为高层策略输入。

5.5 安全指标说明

avoid_metric 反映机器人相对障碍物与边界的安全程度，值越高越安全；reach_metric 反映机器人相对目标的接近程度，值越高越接近目标。二者由底层环境的 _compute_safety_metrics() 计算，是构造 g(s)/h(s) 的直接依据，也是本软件“安全导航”能力的来源。

第六章 操作使用

6.1 训练操作流程

第一步，确认环境与依赖安装正确，CUDA 可用；第二步，在 go2_config.py 中确认场地、障碍、目标等参数符合预期；第三步，运行训练脚本 train_reach_avoid.py 启动训练，控制台将按轮输出成功率与执行成本；第四步，观察日志，待成功率与成本收敛后停止训练，

检查点自动保存至 logs/。

6.3 评测与轨迹导出

运行 test_reach_avoid.py 在大量随机初始条件下批量评测，脚本将每条轨迹的状态、动作与指标写入 JSON 文件。离线可使用该文件统计成功率、碰撞率与平均执行步数（即执行成本），得到可量化的性能结论。

6.2 可视化回放

运行 play_reach_avoid.py 加载已训练的高层策略，仿真将在图形界面中运行。

观察要点：机器人是否朝目标前进、是否在接近障碍物时减速绕行、是否在回合内安全抵达目标。

回放可用于定性判断策略是否符合“安全导航”目标。

6.4 输出文件说明

路径/文件	内容
logs/*.pt	策略与优化器检查点，用于续训与回放
logs/*/training_summary.txt	训练汇总（成功率、成本等）
logs/*.png	训练曲线或场景布局图
test_reach_avoid 导出 JSON	评测轨迹与逐回合指标
output/*.png	用于文档与展示的结果图

6.5 训练曲线解读

典型训练曲线中，成功率随轮数从较低水平（如 0.014）逐步上升至约 0.691，峰值约 0.743；执行成本（平均到达步数）随轮数下降并收敛至约 79.1 步。成功率上升与成本下降通常同步发生，

说明策略在“更快到达”与“更安全”之间取得了平衡。

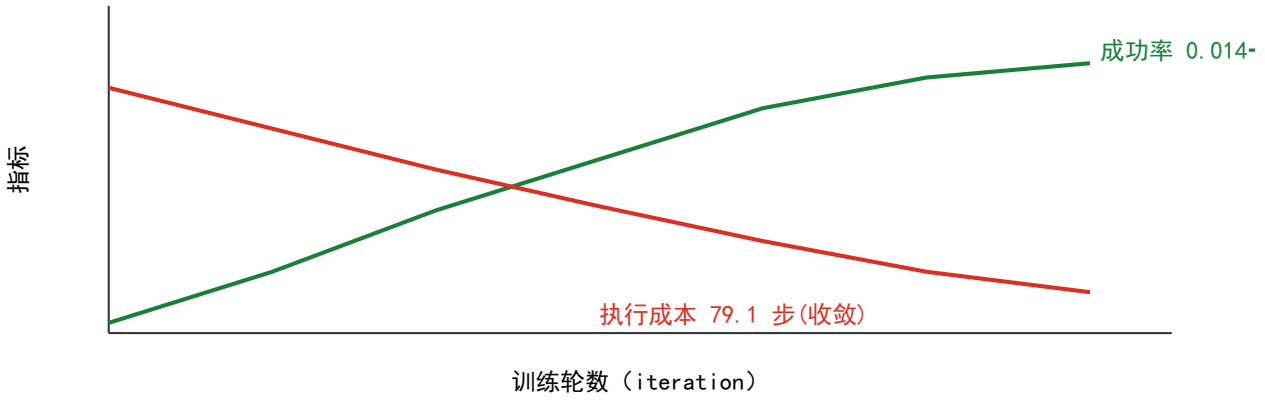


图6-1 典型训练曲线示意（成功率上升、成本收敛）

6.6 实机部署简述

训练所得高层策略为速度命令策略，可经模型导出后在 Unitree Go2 机载算力上运行：

高层输出速度命令，由机载低层控制器（与仿真中冻结策略同构）映射为关节动作。实机部署需额外处理传感器标定、

通信延迟与安全检查，超出本软件仿真范畴，此处仅作接口说明。

第七章 数据库、字典与术语

7.1 数据存储说明

本软件不依赖关系型数据库。训练产出以文件形式保存：策略检查点存于 logs/ 目录（

.pt 文件），训练曲线与布局图以 .png 保存；算法超参与环境参数以 Python

配置类 / 字典形式集中定义，便于版本管理。

7.2 关键参数汇总

参数	含义	取值
g_target	到达目标代价	-300
g_distance_scale	距离缩放	100
h_safe	安全时代价	-300
h_unsafe	不安全时代价	300
obs_dim	高层观测维度	40
act_dim	动作维度	3
field_size	场地尺寸	12×12 m
n_obstacles	障碍球数量	14
target_radius	目标半径	0.5 m
train_iters	训练轮数	1500
success_rate	最终成功率	0.691（峰值0.743）
exec_cost	收敛执行成本	79.1 步

7.3 核心变量字典

名称	含义

g(s)	到达代价函数，抵达目标时取 g_target，距离越远代价越高
h(s)	安全约束函数，安全时取 h_safe，碰撞/越界时取 h_unsafe
avoid_metric	底层环境计算的避障度指标，越高越安全
reach_metric	底层环境计算的到达度指标，越高越接近目标
高层观测 o	40 维向量（基础8+目标lidar16+障碍物lidar16）
速度命令 a	动作 [vx, vy, vyaw]，分别对应前后、左右、偏航
GAE	广义优势估计，用于策略梯度基线

7.4 术语表

现象	可能原因	处理建议
导入报错（找不到 isaacgym）	Isaac Gym 未正确安装	检查是否按 Isaac Gym 官方指引完成安装
CUDA 不可用	驱动/CUDA 版本不匹配	按 PyTorch 官方指引重装对应 CUDA 版本
成功率不上升	初始难度过高/学习率不当	调小障碍数或调学习率，检查安全指标
频繁碰撞	避障权重不足	增大 h_unsafe 或减小目标距离权重
续训失败	检查点路径错误	确认 logs/ 下检查点文件与配置一致

1.6 软件特点小结

综上，本软件的特点可概括为：分层抽象提升样本效率；以 Reach-Avoid 约束统一表达到达与安全；

在底层实时计算 avoid/reach 指标作为安全信号；基于 rsl_rl 的 PPO

框架实现专用算法；训练、回放、评测脚本齐全，便于复现与消融；仿真到实机的接口一致，支持策略部署迁移。

2.4 功能与文件对应

功能	主要文件
仿真环境与安全指标	go2_env.py, _compute_safety_metrics()
高层观测与约束信号	high_level_navigation_env.py
分层时间抽象	hierarchical_go2_env.py
训练算法	reach_avoid_ppo.py, _calculate_reach_gae()
策略/价值网络	actor_critic.py
训练主循环	train_reach_avoid.py, on_policy_runner.py

7.5 常见问题排查

高层策略：本软件的训练对象，输入 40 维观测，输出 3 维速度命令。

对角高斯策略：动作分布为各维独立高斯，仅学习均值与对数标准差，适用于连续动作空间。

Isaac Gym：NVIDIA 提供的并行物理仿真平台，用作本软件仿真后端。

PPO：近端策略优化，本软件训练所基于的 on-policy 强化学习算法。

GAE：广义优势估计，结合时序差分误差估计优势函数，稳定策略梯度。

激光雷达分箱（lidar_bins）：将周围空间按角度分箱并取最近障碍距离，作为观测特征。

Reach-Avoid（到达—避障）：在抵达目标的同时满足安全约束的控制问题，本软件的核心建模方式。

分层强化学习：将决策分解为高层策略与低层策略的多层结构，高层在更粗时间尺度上决策。

冻结低层运动策略：预训练且在训练过程中参数固定的底层控制器，负责稳定行走，隔离不安全动作。

回放与评测	play_reach_avoid.py, test_reach_avoid.py
结果绘图	plot_training_results.py, plot_env_layout.py

接口	维度	含义/范围
观测 obs	40	基础8 + 目标lidar16 + 障碍/边界lidar16
动作 a_t[0] vx	1	前后速度（m/s）
动作 a_t[1] vy	1	左右平移速度（m/s）
动作 a_t[2] vyaw	1	偏航角速度（rad/s）
低层步数/高层步	5	高层每步低层执行 5 步

3.6 性能与适用边界

本项目实际训练中，成功率由约 0.014 提升至 0.691（峰值 0.743），执行成本收敛至约 79.1 步，表明分层安全导航方案有效。其适用边界为：训练在仿真中进行，实机部署需额外处理传感器与延迟；

当前场景为 12×12 米方形场地、14 个障碍球，更大或更复杂场景需相应扩展配置与训练。

4.5 高层策略接口说明

5.6 训练资源与耗时

训练对算力的主要消耗在大规模并行仿真采样与神经网络更新。本项目在单张服务器级 GPU

上完成 1500 轮训练，实际耗时约 21.5 小时；检查点与日志随训练写入磁盘，占用空间主要取决于保存频率与回放画面数量。

若显存或算力有限，可减少并行环境数量，相应延长训练时间。

5.7 配置示例说明

go2_config.py 以 Python 配置类集中管理参数，典型字段包括：场地边长

field_size、障碍球数量 n_obstacles、目标半径 target_radius、

到达代价 g_target 与距离缩放 g_distance_scale、安全代价

h_safe 与 h_unsafe、训练轮数 train_iters 等。修改这些字段即可调整任务难度与约束强度，

无需改动算法代码，体现了配置与逻辑分离的设计。

6.7 评测指标计算方法

成功率 = 在 N 个随机初始回合中安全抵达目标的比例；碰撞率 = 发生碰障或越界的回合比例；

执行成本 = 抵达目标所需的平均仿真步数（步数越少表示路径越短、效率越高）。三者结合可全面衡量“是否到达、

是否安全、是否高效”。test_reach_avoid.py 导出的 JSON 包含逐回合上述指标，

离线汇总即可得到报告。

6.8 结果复现建议

为复现本项目结果，建议：固定随机种子以保证可复现；使用与本手册一致的 go2_config

参数；在相同的 GPU 与 Isaac Gym 版本下训练；保存每轮日志以便绘制曲线。

若成功率偏离较大，优先检查安全指标计算与随机初始化范围是否一致。

7.6 版本与演进

本软件版本为 V1.0.0，为首次登记的完整训练与评测工具集。后续演进方向包括：支持更丰富的地形与动态障碍、引入循环策略处理部分可观信息、以及更完善的实机部署与安全保护组件。版本号在申请表、源代码页眉与操作手册页眉三处保持一致。

7.7 技术支持与声明

本软件以源码形式提供，使用问题可参考随附 README 与 ALGORITHM_DESIGN 文档，或查阅项目展示网页。本手册仅为软件功能与使用的说明，不构成任何担保，涉及论文方法的“剩余预算”“双 Critic”等内容未在本仓库代码中实现，不计入本软著登记范围。

第八章 附录

8.1 源程序文件清单

本软著登记的源程序取自研目录 legged_gym_go2/legged_gym 与 rsl_rl/rsl_rl，下表按模块列出主要源文件及其职责（完整清单以源代码鉴别材料为准）。所有文件均参与训练或评测流程。

文件	职责
rsl_rl/rsl_rl/algorithms/reach_avoid_ppo.py	Reach-Avoid PPO 核心：GAE 与专用缓冲
rsl_rl/rsl_rl/algorithms/ppo.py	基础 PPO 实现（被复用）
rsl_rl/rsl_rl/modules/actor_critic.py	策略与价值网络（MLP 512×4）
rsl_rl/rsl_rl/modules/actor_critic_recurrent.py	带循环单元的网络可选版本
rsl_rl/rsl_rl/storage/rollout_storage.py	轨迹回放缓冲
rsl_rl/rsl_rl/runners/on_policy_runner.py	训练主循环与日志
rsl_rl/rsl_rl/utils/utils.py	通用工具函数
legged_gym_go2/.../envs/go2/go2_config.py	场地/障碍/目标等关键参数
legged_gym_go2/.../envs/go2/go2_env.py	底层环境与安全指标
legged_gym_go2/.../envs/go2/high_level_navigation_env.py	高层观测与 g(s)/h(s)
legged_gym_go2/.../envs/go2/hierarchical_go2_env.py	分层封装（高层1步=低层5步）
legged_gym_go2/.../envs/base/legged_robot.py	足式机器人基类环境
legged_gym_go2/.../envs/base/legged_robot_config.py	基类环境配置
legged_gym_go2/.../envs/base/base_task.py	任务基类
legged_gym_go2/.../envs/base/base_config.py	基础配置
legged_gym_go2/.../envs/base/origin_legged_robot.py	原始足式机器人环境
legged_gym_go2/.../scripts/train_reach_avoid.py	训练主脚本
legged_gym_go2/.../scripts/play_reach_avoid.py	可视化回放
legged_gym_go2/.../scripts/test_reach_avoid.py	批量评测与轨迹导出
legged_gym_go2/.../scripts/play_fixed_commands_with_video.py	定速命令录屏
legged_gym_go2/.../scripts/plot_training_results.py	训练曲线绘图
legged_gym_go2/.../scripts/plot_env_layout.py	场景布局绘图
legged_gym_go2/.../utils/helpers.py	通用辅助
legged_gym_go2/.../utils/terrain.py	地形相关

legged_gym_go2/.../utils/task_registry.py	任务注册
legged_gym_go2/.../utils/logger.py	日志辅助
legged_gym_go2/.../utils/isaacgym_utils.py	Isaac Gym 工具
legged_gym_go2/.../utils/math.py	数学辅助
legged_gym_go2/.../envs/h1_2/*	H1_2 机型环境/配置
legged_gym_go2/.../envs/h1/*	H1 机型环境/配置
legged_gym_go2/.../envs/g1/*	G1 机型环境/配置
legged_gym_go2/.../envs/tinymal/*	TinyML 机型环境/配置

8.2 高层观测分量说明

40 维高层观测的具体分量划分如下，便于理解策略输入含义与调试。

分量区间	含义	维度
o[0:8]	基础特征（位姿/速度/相对目标）	8
o[8:24]	目标方向激光雷达分箱	16
o[24:40]	障碍/边界激光雷达分箱	16

8.3 常用命令示例

训练：运行 train_reach_avoid.py 启动高层策略训练，检查点写入 logs/。

回放：运行 play_reach_avoid.py 加载检查点，在仿真界面观察导航行为。

评测：运行 test_reach_avoid.py 批量评测，将轨迹与指标导出为 JSON 供统计。

绘图：运行 plot_training_results.py 与 plot_env_layout.py 读取日志与配置，生成曲线与布局图。

8.4 提交材料说明

本软著登记需提交三类材料：软件著作权登记申请表（在线填报后打印签章）、源代码鉴别材料（

本手册配套的前30+后30页源代码 PDF，每页不少于 50 行）、文档鉴别材料（即本操作手册，每页不少于 30 行）。源代码页眉与手册页眉的软件全称及版本号须与申请表完全一致。

（本手册完）本手册所述全部内容均依据项目真实源码整理；README 中提及的“剩余预算”“双

Critic”等内容属于论文方法描述，未在本仓库代码中实现，故不写入本手册。如有疑问，以源码材料为准。